Documento de arquitectura

Solución de ingeniería de datos end-to-end · Pólizas y siniestros

Prueba técnica – Ingeniero de Datos · Conexxion Sura · Octubre 2026

Repositorio: github.com/soquendo61-dotcom/conexxion-sura-data-engineering

1. Problema y objetivo

La compañía necesita una vista confiable y trazable de pólizas y siniestros para reportes, tableros, modelos analíticos y consumo operativo. Las fuentes llegan desde sistemas heterogéneos con duplicados, formatos de fecha distintos, claves incompletas, movimientos tardíos y cambios de estado. La solución lleva esas fuentes hasta productos de datos **certificados**, con calidad medida, cuarentena, reconciliación, observabilidad, seguridad y un proceso reproducible de promoción entre ambientes.

2. Diagrama end-to-end



Versión detallada (Mermaid) en `docs/architecture/diagrama_arquitectura.md` del repositorio.

3. Componentes

Componente	Objetivo corporativo (Azure)	Implementación en la prueba (Free Edition)
Fuentes	CRM, core de pólizas y siniestros, pagos, bus de eventos, APIs	5 CSV dummy
Ingesta batch	Azure Data Factory / Lakeflow Connect con CDC o watermark	Archivos en volumen UC + ingesta idempotente por hash (`src/ingestion/bronze.py`)
Ingesta eventos	Event Hubs/Kafka + Auto Loader / Structured Streaming	PoC Auto Loader `availableNow` (`notebooks/10`)

Componente	Objetivo corporativo (Azure)	Implementación en la prueba (Free Edition)
Almacenamiento	ADLS Gen2 por ambiente, Delta Lake, external locations	Tablas Delta **administradas** en `workspace.default`
Procesamiento	Databricks (serverless jobs / job clusters con políticas)	Serverless, PySpark + Spark SQL
Calidad	Data contracts, motor de reglas, cuarentena, reconciliación	`src/contracts`, `src/quality`, `silver_cuarentena`, `gold_calidad_pipeline`
Orquestación	Lakeflow Jobs + Asset Bundles	Workflow de 5 tareas, cron `0 0 6 5 * ?` America/Bogota
Gobierno	Unity Catalog: permisos, linaje, tags, máscaras, auditoría	Funciones `fn_mask_*`, máscaras de columna, tags, comentarios, `GRANT` por rol
Secretos	Azure Key Vault + secret scopes / OIDC	No se requieren secretos; ninguno en código
Observabilidad	Tablas de control + system tables + AI/BI dashboards + alertas	`ctl_*`, `vw_obs_*`, notebook 07, alertas de job
Consumo	Vistas certificadas, Databricks SQL, Power BI, ML, GenAI	`vw_gold_*_certificada`
CI/CD	GitHub + Actions + Environments con aprobación	`.github/workflows/ci.yml`, `cd.yml`, `databricks.yml`

4. Flujo de datos

1. **Aterrizaje.** Los archivos se depositan en `/Volumes/<catalog>/<schema>/volume1/raw/` con el prefijo del dataset.
2. **Bronze** (`tarea bronze`). Se lista cada archivo y se calcula el **SHA-256 de su contenido**:
   - un archivo ya ingerido se omite;
   - cada archivo se lee con su propio encabezado, todo como texto, y las filas malformadas se capturan en `_corrupt_record`;
   - se agregan `_source_file`, `_file_hash`, `_ingestion_ts`, `_run_id`, `_fecha_corte`, `_row_hash` y `_record_id`;
   - se escribe con `replaceWhere _file_hash IN (...)` y `mergeSchema`;
   - el contrato se valida. Si faltan columnas obligatorias, el archivo queda en `PENDIENTE_REVISION` y bloquea la publicación.
3. **Silver** (`tarea silver`). Para cada dataset, procesando primero los padres:
   - aplica el contrato (con alias), limpia y tipa, registrando los valores que no se pueden convertir;
   - marca **duplicados exactos**, elige la **versión vigente** con desempate determinístico y detecta **duplicados de negocio** por huella de contenido;
   - enriquece con integridad referencial y derivados;
   - evalúa las reglas: lo aceptado va a `silver_<dataset>` (MERGE sincronizado) y lo rechazado a `silver_cuarentena`;
   - reconcilia Bronze = Silver + cuarentena + versiones reemplazadas, y construye el SCD2 de clientes.
4. **Gold** (`tarea gold`). Construye `gold_poliza_360`, `gold_siniestralidad_mensual` y `gold_discrepancias` para la `fecha_corte`, escribiendo con `replaceWhere fecha_corte = ...`.
5. **Validación y publicación.** Ejecuta la reconciliación Silver→Gold, las postcondiciones y los controles de frescura. Si todo está OK, registra el corte como **PUBLICADO** en `ctl_publicaciones`, y solo entonces las vistas certificadas lo exponen.
6. **Gobierno y vistas.** Aplica vistas de consumo y observabilidad, máscaras, tags y permisos (idempotente).

5. Decisiones de almacenamiento

Tema	Decisión	Justificación
Formato	Delta Lake	ACID, `MERGE`, `replaceWhere`, evolución de esquema, time travel y Change Data Feed
Tablas	**Administradas** en Unity Catalog	Gobierno completo, *predictive optimization*, sin gestionar rutas. Free Edition no ofrece external locations propias. En el escenario corporativo se usan external locations sobre ADLS cuando el ciclo de vida del dato debe ser independiente del catálogo
Particionamiento	**No se particiona físicamente** (miles de filas)	Particionar tablas pequeñas genera archivos pequeños. Se usa **liquid clustering** por `fecha_corte` (y `periodo`) en Gold, que da *data skipping* y pruning sin el problema de la sobre-partición. Con volúmenes de TB, Bronze se particionaría o agruparía por fecha de ingesta
Granularidad	Bronze: fila física por archivo. Silver: entidad vigente. Gold: póliza por corte y mes × producto × sucursal por corte	Ver `05_modelo_datos_y_calidad.md`
Escritura	Bronze `replaceWhere` por archivo; Silver `MERGE` por clave (actualiza solo si cambia `_silver_hash` y borra las claves que ya no vienen); Gold `replaceWhere` por `fecha_corte`	Idempotencia: reejecutar o reprocesar un periodo no duplica
Archivos pequeños	Lotes por dataset en un solo commit; `OPTIMIZE` y auto-compactación vía predictive optimization; clustering	En la carga corporativa: `optimizeWrite` / `autoCompact` y mantenimiento programado
Historia	Bronze conserva todas las versiones recibidas; Silver mantiene el estado vigente (SCD1); `silver_clientes_historia` en SCD2; Gold guarda una foto por corte	Auditoría y reproceso; consumo simple

6. Estrategia incremental y evolución de fuentes

Fuente	Mecanismo recomendado
Archivos (actual)	Control por **hash de contenido** en `ctl_archivos` más watermark de `file_modification_ts`. Auto Loader (`cloudFiles`) cuando el volumen crece: descubre archivos nuevos con checkpoint y garantiza exactamente una vez
Bases de datos	**CDC** (log-based con Lakeflow Connect, Debezium o ADF CDC) que entrega inserciones, actualizaciones y borrados con número de secuencia. Si no hay CDC, **watermark** por `fecha_actualizacion` con ventana de solape (p. ej. 3 días) para capturar movimientos tardíos, más deduplicación por clave y versión
APIs	Extracción paginada por `updated_since` = último watermark exitoso, guardando el JSON crudo en Bronze
Eventos	Event Hubs/Kafka → Structured Streaming con checkpoint, watermark de evento y `MERGE` por `evento_id` (ver `08_opcionales_streaming_ia.md`)

Watermark: se actualiza **solo después** de una publicación exitosa (`ctl_watermarks`), de modo que un fallo nunca deja datos sin procesar. Silver propaga cambios con `MERGE` y tiene **Change Data Feed** habilitado, para que en el escenario corporativo Gold pueda leer únicamente los cambios (`table_changes`) en lugar de recomputar.

Llegadas tardías: un movimiento de agosto que llega en septiembre se incorpora en el siguiente proceso. Para corregir el corte de agosto se ejecuta un **backfill** del periodo (`fecha_corte = 2026-08-31`, `modo = reproceso`),

que reemplaza solo esa foto de Gold.

7. Separación de ambientes (estrategia corporativa)

Ambiente	Propósito	Datos	Despliegue	Identidad
DEV	Desarrollo y pruebas del equipo	Muestras o datos sintéticos / enmascarados	Automático al fusionar en `dll`	Usuarios + SP dev
QA	Pruebas de integración, regresión y desempeño con contratos reales	Copia enmascarada o subconjunto de producción	Tag `vX.Y.Z`, automático + ejecución de pruebas	SP qa
PROD	Operación	Reales	Mismo artefacto aprobado en QA, **aprobación manual**	SP prod (`run_as`)

- **Aislamiento:** un workspace y un catálogo por ambiente (`dev_seguros`, `qa_seguros`, `prod_seguros`), cada uno con su storage ADLS y su managed identity. Los catálogos se enlazan solo a su workspace (*workspace binding*), de modo que DEV no puede leer PROD.
- **Promoción de código y objetos:** Asset Bundle (`databricks.yml`) con targets. El código no cambia entre ambientes, solo variables (catálogo, schema, correo y estado de la programación). Los objetos UC (tablas, vistas, funciones, permisos) se crean desde código (`resources/sql`) en el despliegue.
- **Controles previos:** PR con revisión obligatoria, CI en verde (lint, pruebas, validación del bundle) y rama `dll` protegida.
- **Aprobaciones:** GitHub Environment `prod` con revisores obligatorios (dueño del producto de datos y líder técnico).
- **Rollback:** redeploy del tag anterior. Para los datos: `RESTORE TABLE ... TO VERSION AS OF n` (time travel) o reproceso del corte (idempotente).
- **Auditoría:** historial de Git/PR, ejecuciones de Actions, `system.access.audit`, historial Delta y `ctl_ejecuciones`.
- **Trade-off:** tres workspaces cuestan más y exigen sincronizar configuraciones, a cambio de aislamiento fuerte, que es lo esperado en el sector asegurador regulado. La alternativa barata (un workspace con catálogos por ambiente) es válida para equipos pequeños.

8. Seguridad y gobierno

- **Clasificación:**

  - PII alta: número de documento, correo y teléfono.
  - PII media: fecha de nacimiento y referencia bancaria.

  Está declarada en los contratos y en tags de Unity Catalog.

- **Protección:**

  - Máscaras de columna (`fn_mask_*`) en `silver_clientes`: solo el grupo `pii_lectores` ve el dato real. La vista `vw_clientes_protegida` es la alternativa portable.
  - Bronze y cuarentena (que contiene el registro original) tienen acceso restringido a ingeniería y data stewards.
  - Gold no expone PII.
- **Mínimo privilegio** (`01_gobierno_unity_catalog.sql`):

Rol	Permisos
`data_engineers`	Administra el schema (en PROD, solo el service principal escribe)

Rol	Permisos
`data_stewards`	Calidad, cuarentena y Silver
`data_analysts`	Solo Gold y vistas certificadas
`pii_lectores`	Desenmascara
Evaluadores	Lectura del schema y del volumen

- **Linaje y auditoría:** Unity Catalog captura el linaje tabla/columna automáticamente. Cada fila Silver/Gold lleva `_record_id`, `_source_file`, `_file_hash` y `run_id`. El historial Delta y `system.access.audit` completan la auditoría.
- **Ownership y documentación:** owner de negocio y técnico por dataset (contratos y tags), comentarios de tabla y columna en UC.
- **Secretos:** la solución no los necesita. Corporativamente se usan Key Vault + secret scopes o OIDC/managed identity, nunca en notebooks ni en Git (CI valida que no haya `.env` ni `.databrickscfg`).
- **Retención y datos personales** (Ley 1581 de 2012, habeas data):
  - Retención de Bronze limitada (p. ej. 13 meses) con `VACUUM` y una política de retención de logs Delta acorde.
  - Silver/Gold según la regulación contable y aseguradora.
  - Derecho de supresión: borrado por `cliente_id` en Bronze/Silver con `DELETE` + `VACUUM` (y *deletion vectors*), registrado en una bitácora de solicitudes.
  - Minimización: Gold solo contiene atributos no identificables.

9. Observabilidad

Ver 06_operacion_observabilidad.md. Incluye estado y duración por tarea, volúmenes de entrada y salida, aceptados y rechazados, duplicados, evolución de esquema, frescura y SLA, costo estimado, resultado de cada regla y alertas del job.

10. Riesgos, decisiones y trade-offs

Riesgo / decisión	Mitigación / razón
Fechas `dd/MM/yyyy` ambiguas con `MM/dd`	Supuesto explícito: día/mes (convención colombiana). Se debe ratificar con el productor e incluir en el contrato
Alto % de pagos (49%) y siniestros (79%) fuera de vigencia	No se tratan como error bloqueante: son discrepancias de negocio publicadas en `gold_discrepancias` con umbrales altos de advertencia. Probable artefacto del dato dummy
Estado de la póliza sin historia en la fuente	El backfill usa el estado vigente. El SCD2 de pólizas a partir de los movimientos queda como deuda técnica
Recomputar Silver desde Bronze en cada corrida	Simple y determinístico para este volumen. A escala: incremental con watermark/CDF (diseñado)
Fecha efectiva de cancelación	Se toma `fecha_actualizacion` si cae dentro de la vigencia; si no, `fin_vigencia`. Debe validarse con negocio
Umbrales de calidad	Calibrados con el perfilamiento; deben acordarse con los owners en el data contract
Free Edition	Un ambiente, serverless y grupos limitados. Lo no soportado se reporta como `NO_APLICADA` y se documenta